

Assignment: ODE-IVP

Name: 유채은

ID: 22200476

Academic Integrity Declaration

Check the statement that applies and sign your name.

Check	Statement
V	I have NOT used LLM (ChatGPT, etc), any online materials, or other students' reports/codes to complete this assignment.
	I have NOT completed this assignment entirely on my own.

If you received any assistance **that violates the class policy**, you must explain it in detail. A penalty will be applied depending on the **extent and nature of the assistance**.

Failure to disclose the assistance, you will get Grade F.

Examples:

- Screenshots of LLM prompt,
- Explanation of assistance : who/how/what extent etc

By signing below, I confirm that I have honestly and truthfully answered the academic integrity declaration.

Date: 2025.11.17

Sign: 유채은

What you need to submit

Submit as a zip file: [Assignment_ODE_YuChaeun_22200476.zip](#)

- **Report** (pdf)
- **Src Codes:** (1) [Assignment_ODE_YuChaeun_22200476.cpp](#)
 (2) [myNP_22200476.h](#), [myNP_22200476.cpp](#)
 (3) [myMatrix_22200476.h](#), [myMatrix_22200476.cpp](#)
- **Download template code:** [Assignment_ODE_student.cpp](#)

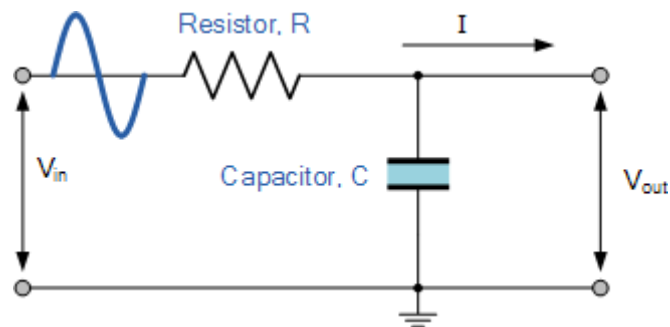
Part 0: Tutorial

Review Tutorial: [TU ODE IVP](#)

Part 1: 1st order ODE

Problem 1

Solve for the response (V_{out}) of an RC circuit with a sinusoidal input (V_{in}), from $t=0$ to 0.1 sec.



$$f(t, v) = \frac{dv}{dt} = -\frac{1}{\tau} v(t) + \frac{1}{\tau} V_m \cos(2\pi ft)$$

where

$$\tau=0.01, \quad f=100\text{Hz}, \quad V_m=1 \text{ [V]}, \quad h=0.001$$

Procedure

a) Write down a pseudocode for 2nd order Runge-Kutta method

```
N = (tf - t0) / h + 1

y[0] = y_init

// RK Design Parameters
C1 = 0.5
C2 = 1 - C1
alpha = 1
beta = alpha

// RK2 ODE Solver
for i = 1 to N

    K1 = odeFunc(ti, y(i))
    K2 = odeFunc(ti + alpha * h, y(i) + beta * K1 * h)

    // Estimate: y(i+1) using RK2
    Y(i + 1) = y(i) + (C1 K1 + C2 K2) h

    ti += h
end i
```

b) Create C/C++ function for RC system ODE function

```
double odeFunc_rc(const double t, const double v)
```

```
double odeFunc_rc(const double t, const double v)
{
    // Input:    y, t
    // Output:   dydt

    // system modeling parameters
```

```

double tau = 0.01;
double f = 100;
double Vm = 1;
double omega = 2 * PI * f;
double dvdt = 0;

// output
dvdt = -(1 / tau) * v + (1 / tau) * cos(omega * t);

return dvdt;
}

```

c) Create C/C++ function for 2nd order Runge-Kutta method

```

void odeRK2(double y[], double odeFunc(const double t, const double y),
            const double t0, const double tf, const double h, const double y_init)

```

For RK2, the default value is alpha=1, C1=0.5

```

void odeRK2(double y[], double odeFunc(const double t, const double y), const double t0,
const double tf, const double h,
const double y_init)
{
    // Variable Initialization
    int N = (tf - t0) / h + 1;
    double y2 = 0;
    double t2 = 0; // 중간(임시) y

```

```

// Initial Condition
double ti = t0;
y[0] = y_init;

// RK Design Parameters
double C1 = 0.5;
double C2 = 1 - C1;
double alpha = 1;
double beta = alpha;
double K1 = 0, K2 = 0;

// RK2 ODE Solver
for (int i = 0; i < N - 1; i++) {

    // [First-point Gradient]
    K1 = odeFunc(ti, y[i]);

    // [Second-point Gradient]
    t2 = ti + alpha * h;

    y2 = y[i] + beta * K1 * h;

    K2 = odeFunc(t2, y2);

    // Estimate: y(i+1) using RK2
    y[i + 1] = y[i] + (C1 * K1 + C2 * K2) * h;

    ti += h;
}
}

```

See the given assignment template code.

c) Create C/C++ function for 3rd order Runge-Kutta method

```

void odeRK3 (double y[], double odeFunc(const double t, const double y),
             const double t0, const double tf, const double h, const double y_init)

```

For RK3, use classical third-order Runge-Kutta

```

void odeRK3(double y[], double odeFunc(const double t, const double y), const double t0,
const double tf, const double h, const double y0)
{
    // Variable Initialization
    int N = (tf - t0) / h + 1;
    double y2, y3 = 0;
    double t2, t3 = 0;

    // Initial Condition
    double ti = t0;
    y[0] = y0;

    // RK Design Parameters
    double C1 = 1.0 / 6.0;
    double C2 = 4.0 / 6.0;
    double C3 = 1.0 / 6.0;
    double alpha2 = 0.5, alpha3 = 1;
    double beta21 = 0.5, beta31 = -1, beta32 = 2.0;
    double K1 = 0, K2 = 0, K3 = 0;

    // RK2 ODE Solver
    for (int i = 0; i < N - 1; i++) {

        // [First-point Gradient]
        K1 = odeFunc(ti, y[i]);

        // [Second-point Gradient]
        // Calculate t3, y3, and K3
        t2 = ti + alpha2 * h;
        y2 = y[i] + beta21 * K1 * h;
        K2 = odeFunc(t2, y2);

        t3 = ti + alpha3 * h;
        y3 = y[i] + beta31 * K1 * h + beta32 * K2 * h;
        K3 = odeFunc(t3, y3);

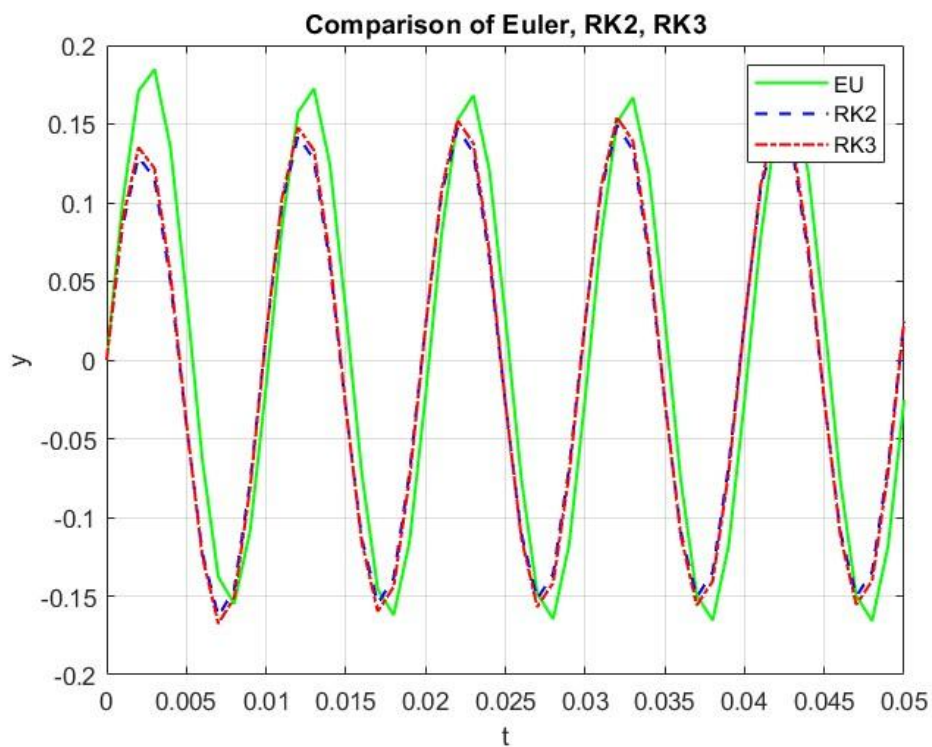
        // Estimate: y(i+1) using RK2
        y[i + 1] = y[i] + (C1 * K1 + C2 * K2 + C3 * K3) * h;

        ti += h;
    }
}

```

d) Compare your answer with MATLAB ODE solver by copy-pasting your outputs in MATLAB and plotting results on the same graph.

1st ODE - IVP Results				
i	t	yEU	yRK2	yRK3
0	0.00000	0.00000	0.00000	0.00000
1	0.00100	0.08090	0.08545	0.08888
2	0.00200	0.10371	0.12919	0.13504
3	0.00300	0.06244	0.11537	0.12170
4	0.00400	-0.02471	0.05005	0.05470
5	0.00500	-0.12223	-0.04111	-0.03968
6	0.00600	-0.19091	-0.12265	-0.12478
7	0.00700	-0.20272	-0.16286	-0.16753
8	0.00800	-0.15155	-0.14584	-0.15110
9	0.00900	-0.05549	-0.07763	-0.08130



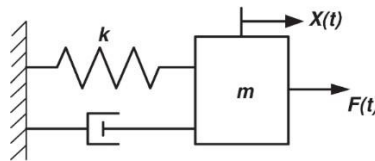
Part 2: 2nd order ODE

Problem 2

A mass-spring-damper (m-c-k) system is a 2nd order ODE

$$m \frac{d^2 y}{dt^2} + c \frac{dy}{dt} + ky(t) = F(t)$$

where $F(t)$ is the input force, $y(t)$ is the displacement.



Create an ODE program that solves for the values of $y(t)$, $dy/dt(t)$

Let the input force $F(t) = A \cos(2\pi ft)$, where

- $m=1\text{kg}$, $k=6.9\text{N/m}$, $c=7\text{ N/m/s}$, $A=2\text{ N}$, $f=5\text{Hz}$,
- $t=0\text{ to }1\text{ sec}$, $h=0.01\text{ sec}$, $y(0)=0.0\text{ m}$, $dy/dt|_{t=0}=0.2\text{ m/s}$

HINT:

$$\begin{cases} \dot{y} = z(t) \\ y = z = \frac{1}{m}(-ky(t) - cz(t) + u(t)) \end{cases} \rightarrow \begin{bmatrix} y \\ \dot{z} \end{bmatrix} = \begin{bmatrix} z(t) \\ \frac{1}{m}(-ky(t) - cz(t) + u(t)) \end{bmatrix}$$

Procedure

a) Write a pseudocode of RK2 for 2nd order system

```
// Variable Initialization
```

```
N = (tf - t0) / h + 1;
```

```
// Initial Condition
```

```
y1[0] = y1_init;
```

```
y2[0] = y2_init;
```

```
// RK Design Parameters
```

```
C1 = 0.5
```

```
C2 = 1 - C1
```

```
alpha = 1
```

```
beta = alpha
```

```
// RK2 ODE for 2nd ODE
```

```
for i = 1 to N
```

```
    yi[0] = y1[i];
```

```
    yi[1] = y2[i];
```

```
    odeFuncSys(K1, ti, yi);    // slope K1 @ current y(i) and v(i)
```

```
    t2 = ti + alpha * h;
```

```
    yE[0] = y1[i] + beta * K1[0] * h
```

```
    yE[1] = y2[i] + beta * K1[1] * h;
```

```
    odeFuncSys(K2, t2, yE);    // slope K2 @ the next y(i+1) and v(i+1)
```

```
    y1[i + 1] = y1[i] + (C1 * K1[0] + C2 * K2[0]) * h;
```

```
    y2[i + 1] = y2[i] + (C1 * K1[1] + C2 * K2[1]) * h;
```

```
    ti += h;
```

```
}
```

b) Create a function of m-c-k ODE function

```
void odeFunc_mck(double dYdt[], const double t, const double Y[]);
```

```

// Input:      vecY = [y; v], t
// Output:     dYdt = [dydt; dvdt]

// system modeling parameters
double m = 1.0;
double c = 7.0;
double k = 6.9;
double A = 2.0;
double f = 5;
double omega = 2 * PI * f;

// output dYdt[0] = ydot
dYdt[0] = Y[1];

// dYdt[1] = vdot
dYdt[1] = 1 / m * (-k * Y[0] - c * Y[1] + A * cos(omega * t));

```

c) Create a function of RK2 for 2nd order system

```

void sys2RK2(double y1[], double y2[],
             void odeFuncSys(double dYdt[], const double t, const double Y[]), const
             double t0, const double tf, const double h, const double y1_init, const double
             y2_init);

```

// Variable Initialization

```

int N = (tf - t0) / h + 1;
double ti = t0;
double t2; // 중간 임시 t

```

// Initial Condition

```

y1[0] = y1_init; // _1 = y
y2[0] = y2_init; // _2 = z ... 나는 인덱스 0->y, 1->v 하고싶은데 여기서 y1은 y, y2는 v

```

줘버려서 따로 다시 정의해줘야함

// RK Design Parameters

```

double C1 = 0.5;
double C2 = 1 - C1;
double alpha = 1;
double beta = alpha;
double K1[2] = { 0 };
double K2[2] = { 0 };

```

```

double yi[2] = { 0 }; // yi[0] = 현재y(i), yi[1] = 현재v(i)

```

```

double yE[2] = { 0 }; // yE[0] = 임시y(i+1), yE[1] = 임시v(i+1)

```

```

// RK2 ODE for 2nd ODE
for (int i = 0; i < N - 1; i++) {

    yi[0] = y1[i];      // current y
    yi[1] = y2[i];      // current v

    odeFuncSys(K1, ti, yi);    // slope K1 @ current y(i) and v(i)

    t2 = ti + alpha * h;

    yE[0] = y1[i] + beta * K1[0] * h
    yE[1] = y2[i] + beta * K1[1] * h;

    odeFuncSys(K2, t2, yE);    // slope K2 @ the next y(i+1) and v(i+1)

    y1[i + 1] = y1[i] + (C1 * K1[0] + C2 * K2[0]) * h;
    y2[i + 1] = y2[i] + (C1 * K1[1] + C2 * K2[1]) * h;

    ti += h;
}

```

c) Compare the answer with MATLAB's ODE solver by copy-pasting your outputs in MATLAB and plotting results on the same graph.

2nd order ODE - IVP Results			
i	t	Y	V
0	0.00000	0.00000	0.20000
1	0.01000	0.00203	0.20523
2	0.02000	0.00410	0.20810
3	0.03000	0.00619	0.20710
4	0.04000	0.00825	0.20118
5	0.05000	0.01022	0.18985
6	0.06000	0.01205	0.17319
7	0.07000	0.01368	0.15187
8	0.08000	0.01508	0.12709
9	0.09000	0.01622	0.10043
10	0.10000	0.01709	0.07368
11	0.11000	0.01770	0.04873

